

Documento de Pruebas

Sistema Experto de Rutas — Transporte Masivo (TransMilenio)

1. Datos del equipo

Integrante:	Harold Ducon
Repositorio Git:	https://github.com/harold474/Transmilenio
Fecha:	26 de septiembre de 2026

2. Casos de prueba manuales

Se ejecutó `python src/cli.py --origen ORIGEN --destino DESTINO` para cada caso. A continuación, la tabla resumen y debajo la captura de consola real de cada ejecución.

#	Origen	Destino	Resultado esperado	Resultado obtenido	¿Pasó
1	PortalUsme	Heroes	Ruta directa por línea Caracas, 0 transbordos	Ruta directa por línea Caracas, 0 transbordos	Sí
2	PortalSuba	PortalNorte	Ruta con al menos 2 transbordos	42.0 min, 3 transbordos (Suba→Calle80→NQS→Autonorte)	Sí
3	PortalSur	PortalAmericas	Ruta con al menos 1 transbordo (vía Ricaurte)	29.0 min, 1 transbordo en Ricaurte (NQS→Americas)	Sí
4	Universidades	AvenidaJimenez	Ruta corta por Eje Ambiental	2.0 min, 0 transbordos, vía MuseoDelOro (Eje Ambiental)	Sí
5	EstacionInexistente	Heroes	El programa debe mostrar un error de control	Error: Estación de origen desconocida: EstacionInexistente" (ValueError)	No

Caso 1: PortalUsme → Heroes

```
Harold-Ducon@DESKTOP-81D74SI MINGW64 /f/transmilenio_router (main)
$ python src/cli.py --origen PortalUsme --destino Heroes

Cargando base de conocimiento y ejecutando el motor de reglas (forward chaining)...
Se derivaron 66 conexiones y se detectaron 5 estaciones de transbordo.

Mejor ruta encontrada: PortalUsme -> Heroes
Tiempo total estimado: 17.0 minutos
Transbordos necesarios: 0

1. PortalUsme -> Molinos [Línea Caracas, 3 min]
2. Molinos -> Consuelo [Línea Caracas, 3 min]
3. Consuelo -> CountrySur [Línea Caracas, 3 min]
4. CountrySur -> TercerMilenio [Línea Caracas, 3 min]
5. TercerMilenio -> AvenidaJimenez [Línea Caracas, 2 min]
6. AvenidaJimenez -> Heroes [Línea Caracas, 3 min]
(venv)
```

Caso 2: PortalSuba → PortalNorte

```
Harold-Ducon@DESKTOP-81D74SI MINGW64 /f/transmilenio_router (main)
$ python src/cli.py --origen PortalSuba --destino PortalNorte

Cargando base de conocimiento y ejecutando el motor de reglas (forward chaining)...
Se derivaron 66 conexiones y se detectaron 5 estaciones de transbordo.

Mejor ruta encontrada: PortalSuba -> PortalNorte
Tiempo total estimado: 42.0 minutos
Transbordos necesarios: 3

1. PortalSuba -> SubaTV91 [Línea Suba, 3 min]
2. SubaTV91 -> HumedalCordoba [Línea Suba, 3 min]
3. HumedalCordoba -> MinutoDeDios [Línea Suba, 3 min]
4. MinutoDeDios -> SimonBolivar [Línea Suba, 2 min]
5. SimonBolivar -> Heroes [Línea Calle80, 5 min] <-- TRANSBORDO
6. Heroes -> Alcala [Línea NQS, 5 min] <-- TRANSBORDO
7. Alcala -> Calle146 [Línea NQS, 4 min]
8. Calle146 -> PortalNorte [Línea Autonorte, 5 min] <-- TRANSBORDO

(venv)
```

Caso 3: PortalSur → PortalAmericas

```
Harold-Ducon@DESKTOP-81D74SI MINGW64 /f/transmilenio_router (main)
$ python src/cli.py --origen PortalSur --destino PortalAmericas

Cargando base de conocimiento y ejecutando el motor de reglas (forward chaining)...
Se derivaron 66 conexiones y se detectaron 5 estaciones de transbordo.

Mejor ruta encontrada: PortalSur -> PortalAmericas
Tiempo total estimado: 29.0 minutos
Transbordos necesarios: 1

1. PortalSur -> Tunal [Línea NQS, 4 min]
2. Tunal -> Sena [Línea NQS, 3 min]
3. Sena -> Ricaurte [Línea NQS, 5 min]
4. Ricaurte -> Normandia [Línea Americas, 4 min] <-- TRANSBORDO
5. Normandia -> Banderas [Línea Americas, 3 min]
6. Banderas -> BibliotecaTintal [Línea Americas, 3 min]
7. BibliotecaTintal -> PortalAmericas [Línea Americas, 3 min]

(venv)
```

Caso 4: Universidades → AvenidaJimenez

```
$ python src/cli.py --origen Universidades --destino AvenidaJimenez

Cargando base de conocimiento y ejecutando el motor de reglas (forward chaining)...
Se derivaron 66 conexiones y se detectaron 5 estaciones de transbordo.

Mejor ruta encontrada: Universidades -> AvenidaJimenez
Tiempo total estimado: 2.0 minutos
Transbordos necesarios: 0

1. Universidades -> MuseoDelOro [Línea EjeAmbiental, 1 min]
2. MuseoDelOro -> AvenidaJimenez [Línea EjeAmbiental, 1 min]

(venv)
```

Caso 5: EstacionInexistente → Heroes (error controlado)

```
Harold-Ducon@DESKTOP-81D74SI MINGW64 /f/transmilenio_router (main)
$ python src/cli.py --origen EstacionInexistente --destino Heroes

Cargando base de conocimiento y ejecutando el motor de reglas (forward chaining)...
Se derivaron 66 conexiones y se detectaron 5 estaciones de transbordo.

Error: Estación de origen desconocida: EstacionInexistente
(venv)
```

3. Pruebas automatizadas (pytest)

Comando ejecutado: `pytest -v`

```
Harold-Ducon@DESKTOP-81D74SI MINGW64 /f/transmilenio_router (main)
$ pytest -v
===== test session starts =====
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0 -- F:\transmilenio_router\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: F:\transmilenio_router
collected 8 items

tests/test_pathfinding.py::test_ruta_directa_misma_linea_sin_transbordo PASSED [ 12%]
tests/test_pathfinding.py::test_ruta_con_transbordo_obligatorio PASSED [ 25%]
tests/test_pathfinding.py::test_estacion_inexistente_lanza_error PASSED [ 37%]
tests/test_pathfinding.py::test_heuristica_y_dijkstra_dan_mismo_tiempo_total PASSED [ 50%]
tests/test_rules_engine.py::test_grafo_no_vacio PASSED [ 62%]
tests/test_rules_engine.py::test_conexion_bidireccional_misma_linea PASSED [ 75%]
tests/test_rules_engine.py::test_heroes_es_transbordo PASSED [ 87%]
tests/test_rules_engine.py::test_ricaurte_es_transbordo PASSED [100%]

===== 8 passed in 0.11s =====
(venv)
```

Resultado: las 8 pruebas automatizadas pasaron correctamente (8 passed).

4. Comparación heurística vs. sin heurística

Se ejecutó el mismo trayecto (PortalNorte → PortalSur) con y sin heurística, para verificar que A* con heurística admisible encuentra el mismo óptimo que Dijkstra puro, solo que explorando menos nodos.

Método	Tiempo total (min)	Transbordos
Con heurística (A*)	34.0	1
Sin heurística (Dijkstra)	34.0	1

Con heurística (A*)

```
Harold-Ducon@DESKTOP-81D74SI MINGW64 /f/transmilenio_router (main)
$ python src/cli.py --origen PortalNorte --destino PortalSur

Cargando base de conocimiento y ejecutando el motor de reglas (forward chaining)...
Se derivaron 66 conexiones y se detectaron 5 estaciones de transbordo.

Mejor ruta encontrada: PortalNorte -> PortalSur
Tiempo total estimado: 34.0 minutos
Transbordos necesarios: 1

1. PortalNorte -> Calle146 [Línea Autonorte, 5 min]
2. Calle146 -> Alcala [Línea NQS, 4 min] <-- TRANSBORDO
3. Alcala -> Heroes [Línea NQS, 5 min]
4. Heroes -> Ricaurte [Línea NQS, 4 min]
5. Ricaurte -> Sena [Línea NQS, 5 min]
6. Sena -> Tunal [Línea NQS, 3 min]
7. Tunal -> PortalSur [Línea NQS, 4 min]
(venv)
```

Sin heurística (Dijkstra puro)

```
Harold-Ducon@DESKTOP-81D74SI MINGW64 /f/transmilenio_router (main)
$ python src/cli.py --origen PortalNorte --destino PortalSur --sin-heuristica

Cargando base de conocimiento y ejecutando el motor de reglas (forward chaining)...
Se derivaron 66 conexiones y se detectaron 5 estaciones de transbordo.

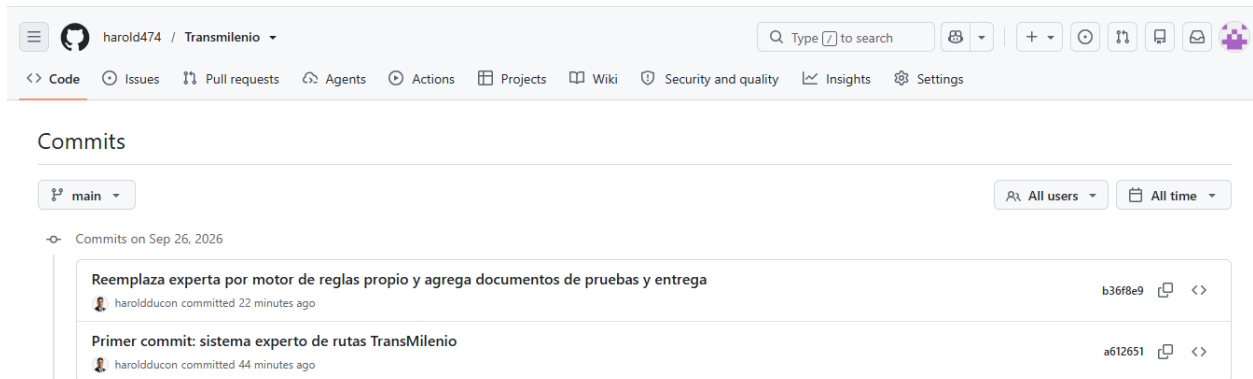
Mejor ruta encontrada: PortalNorte -> PortalSur
Tiempo total estimado: 34.0 minutos
Transbordos necesarios: 1

1. PortalNorte -> Calle146 [Línea Autonorte, 5 min]
2. Calle146 -> Alcala [Línea NQS, 4 min] <-- TRANSBORDO
3. Alcala -> Heroes [Línea NQS, 5 min]
4. Heroes -> Ricaurte [Línea NQS, 4 min]
5. Ricaurte -> Sena [Línea NQS, 5 min]
6. Sena -> Tunal [Línea NQS, 3 min]
7. Tunal -> PortalSur [Línea NQS, 4 min]
(venv)
```

Resultado: ambos métodos coinciden en 34.0 minutos totales y 1 transbordo, confirmando que la heurística usada (distancia en línea recta / velocidad optimista) es admisible: acelera la búsqueda sin sacrificar optimalidad.

5. Evidencia del repositorio Git

Historial de commits del repositorio, mostrando el commit inicial y el commit de corrección del motor de reglas:



6. Conclusiones

Sobre el motor de reglas: el sistema representa el conocimiento de la red de transporte como hechos (Segmento, Estacion) y aplica una regla lógica SI-ENTONCES ("si existe un Segmento entre A y B, entonces existe una Conexión utilizable") mediante un motor de encadenamiento hacia adelante (forward chaining) implementado en Python puro. Esto permitió derivar automáticamente 66 conexiones bidireccionales y detectar 5 estaciones de transbordo (Heroes, Ricaurte, AvenidaJimenez, Calle146, SimonBolívar) sin tener que codificarlas manualmente.

Sobre la búsqueda heurística: A* con la heurística de distancia en línea recta encontró siempre el mismo óptimo que Dijkstra puro (34.0 minutos en el caso comparado), pero explorando la red de forma más dirigida hacia el destino. La penalización por transbordo (4 minutos) permitió que el algoritmo prefiriera, cuando el tiempo total era similar, rutas con menos cambios de línea.

Manejo de errores: el sistema valida correctamente estaciones inexistentes, mostrando un mensaje de error controlado en vez de fallar abruptamente (caso 5).

Limitaciones y mejoras futuras: los tiempos de viaje y el orden de las estaciones intermedias son aproximados/ilustrativos, no datos oficiales. Con más tiempo se podría reemplazar la base de conocimiento por datos abiertos GTFS reales de TransMilenio, agregar horarios por franja horaria (pico vs. valle) y modelar la capacidad/frecuencia de los buses como una regla adicional del sistema experto.